

Relatório MC613

Lab 09 - Periférico Matricial

Prof. Dr. Isaías Felzmann

Ana Beatriz Mota Ferreira (248220)
Ana Beatriz Barbosa Yoshida (245609)
Sofia Valverde Villas Bôas (252974)

1. Introdução

Esse relatório detalha o projeto e a implementação de um acelerador de hardware para aplicação do filtro de Sobel em imagens digitais. O sistema foi desenvolvido para a FPGA e é controlado por um script em Python operando em um computador host. A comunicação entre o computador host e o acelerador é realizada através de uma interface JTAG-UART.

Modularizamos o projeto em algumas etapas principais descritas no diagrama anexado na entrega. Neste documento, iremos abordar informações relevantes sobre cada uma delas.

2. Arquitetura

O planejamento inicial, conforme o diagrama de arquitetura, previa uma implementação baseada em *streaming* com *line buffers*. Esta abordagem visava otimizar o uso de memória e permitir o processamento de imagens de grande porte.

No entanto, para a presente implementação, optamos por uma arquitetura de **Frame Buffer**. Nesta abordagem, a imagem completa recebida do host é primeiro armazenada em um único bloco de memória BRAM na FPGA. Somente após o recebimento de todos os pixels, a fase de processamento é iniciada. Realizamos essa escolha para simplificar a FSM de controle e com o objetivo de validar o protocolo de comunicação e o fluxo de dados de ponta a ponta.

Além disso, vale mencionar que após a etapa de planejamento configuramos a UART virtual de acordo com o passo a passo do enunciado.

3. Implementação no Host (Script Python)

O script Python gerencia a interação do usuário com o acelerador. Suas principais responsabilidades são:

- **Pré-processamento:** Carrega uma imagem, converte-a para escala de cinza, redimensiona para um tamanho máximo de 255x255 e aplica um padding de 1 pixel para que o kernel 3x3 possa operar nas bordas.

- **Comunicação:** Utiliza a biblioteca *intel_jtag_uart* para se comunicar com a FPGA.
- **Transmissão e Recepção:** Segue o protocolo definido, enviando o cabeçalho, as dimensões e os pixels, e depois aguardando a resposta para receber e reconstruir a imagem resultante.
- **Visualização:** Utiliza matplotlib para exibir a imagem original (com padding) e a imagem processada pela FPGA lado a lado, permitindo uma verificação visual do resultado.

4. Implementação na FPGA (Verilog)

O design na FPGA é modular e composto por três partes principais:

4.1. Módulo uart

Este módulo serve como abstrai o IP Core JTAG UART gerado pelo Platform Designer do quartus. Com isso, temos uma interface síncrona e simples para o resto do sistema, semelhante ao utilizado em laboratórios passados da matéria. Utilizamos aqui o exemplo dado pelo professor como base.

4.2. Módulo controlador (Top-Level)

Este é o módulo principal que contém a lógica de controle central.

Memórias: Duas buffers dentro da BRAM de 64KB (*imagem_in* e *imagem_out*) são instanciadas. A primeira armazena a imagem original recebida do host, enquanto a segunda armazena a imagem processada pelo acelerador. O uso de duas memórias separadas teve como objetivo evitar conflitos de leitura e escrita durante a fase de processamento.

FSM: Orquestra todo o processo de forma sequencial, passando pelos seguintes estados:

IDLE: Aguarda o recebimento do cabeçalho 0xAA.

RECEBE_ALTURA e RECEBE_LARGURA: Captura e armazena as dimensões da imagem.

RECEBE_PIXELS: Loop que lê cada pixel da UART e o armazena sequencialmente na memória imagem_in.

PROCESSA_START e PROCESSA_WAIT: Ativa o módulo acelerador e aguarda a sua conclusão, monitorando o sinal done.

ENVIA_HEADER e ENVIA_PIXELS: Após o processamento, envia o cabeçalho de resposta e transmite sequencialmente os pixels da memória imagem_out de volta para o host.

4.3. Módulo acelerador

Operação: Ao receber o sinal de start do controlador, ele inicia um loop que varre todos os pixels internos da imagem (ignorando as bordas).

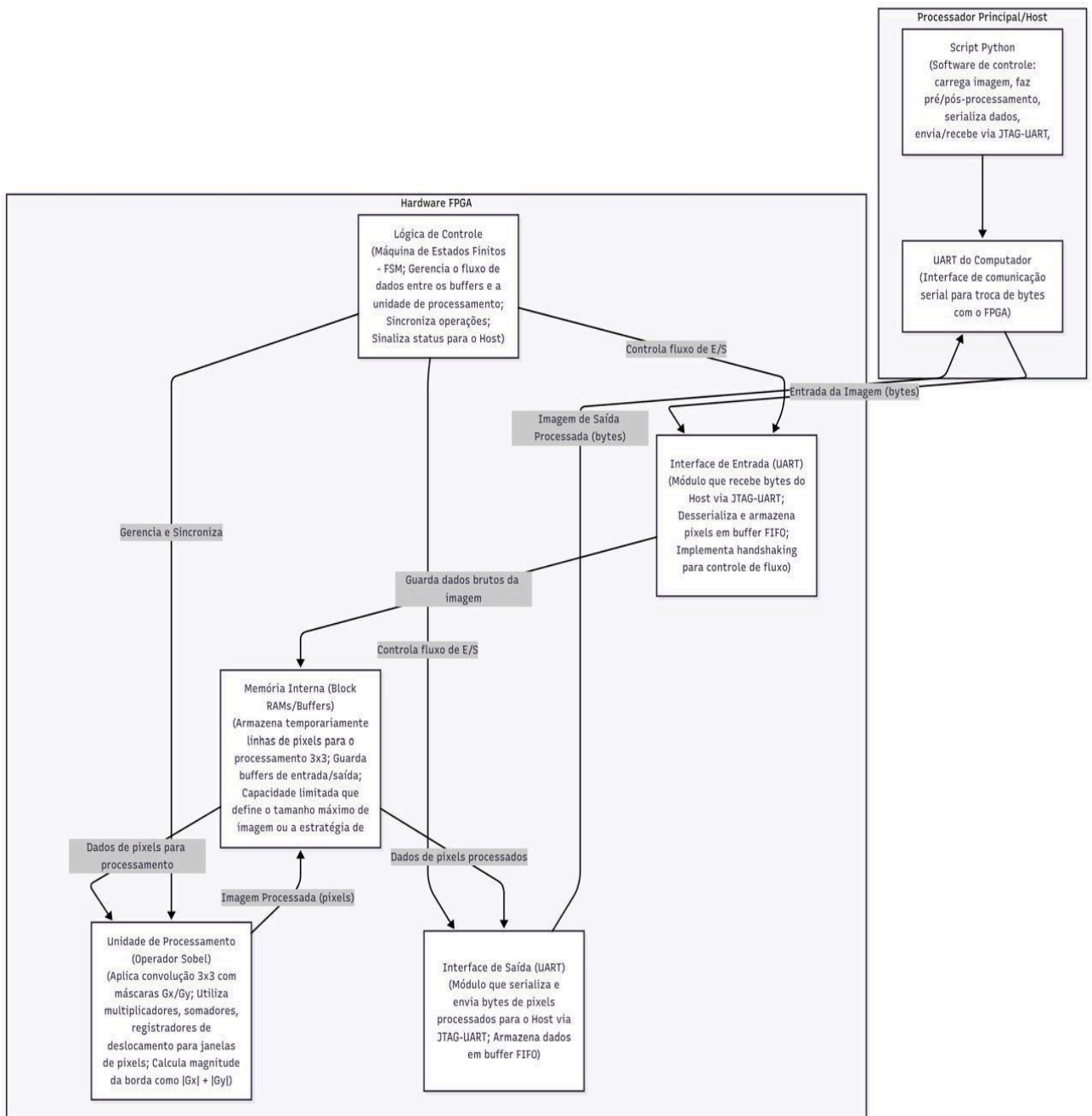
Leitura da Janela: Para cada pixel de destino (y, x), o acelerador lê sequencialmente os 9 pixels que formam a janela 3x3 ao seu redor da memória imagem_in.

Cálculo do Gradiente: Aplica os kernels Sobel Gx e Gy. Para otimizar o uso de recursos de hardware e evitar a necessidade de multiplicadores e raiz quadrada, a magnitude do gradiente é aproximada pela soma dos valores absolutos: $G \approx |Gx| + |Gy|$ [1] [2]. O resultado é saturado em 255.

Escrita do Resultado: O pixel de saída calculado é escrito na memória imagem_out.

Finalização: Após processar todos os pixels, o módulo ativa o sinal done para notificar o controlador.

5. Diagrama de Sequência



6. Conclusão

O projeto de um acelerador de hardware para o filtro de Sobel foi desenvolvido, desde a definição da arquitetura e do protocolo de comunicação até a codificação dos módulos em Verilog e do script de controle em Python. A arquitetura de Frame Buffer foi escolhida e implementada, especialmente pela simplicidade desta, e os componentes necessários para o funcionamento do sistema foram criados.

Embora a validação final na placa não tenha sido concluída, o processo de design e implementação serviu como um exercício de aprendizado profundo para o nosso grupo.

Como trabalho futuro, o próximo passo seria depurar a integração final na placa para obter o sistema funcional. Subsequentemente, gostaríamos de refatorar o design para uma arquitetura de streaming com line buffers. Essa arquitetura eliminaria a necessidade de armazenar a imagem inteira, reduziria o consumo de BRAM e permitiria o processamento de um pixel de saída a cada ciclo de clock (após uma latência inicial), resultando em uma aceleração significativamente maior e representando um avanço substancial na eficiência do acelerador.

7. Referências

- [1] https://docs.opencv.org/4.x/d2/d2c/tutorial_sobel_derivatives.html
- [2] https://www.ic.unicamp.br/~helio/disciplinas/MO443/aula_segmentacao.pdf
- [3] <https://www.intel.com/content/www/us/en/docs/programmable/683130/21-4/jtag-uart-core.html>
- [4] <https://tomverbeure.github.io/2021/05/02/Intel-JTAG-UART.html>
- [5] https://tomverbeure.github.io/2021/05/08/Write-Your-Own-C-and-Python-Clients-for-Intel-JTAG-UART-with-libjtag_atlantic.html